Startup-Scope-AI: An AI-Powered Platform for Startup Idea Evaluation and Founder Support

Project submitted to the

SRM University – AP, Andhra Pradesh

for the partial fulfillment of the requirements to award the degree of

Bachelor of Technology

In

Computer Science and Engineering

School of Engineering and Sciences

Submitted by

A. Siva Sai ManiKanta AP23110010819

Ragolu Nagesh AP23110010680

K. Bharath Kumar AP23110010835

Under the Guidance of
Rithvik Konakala

SRM University–AP

Neerukonda, Mangalagiri, Guntur

Andhra Pradesh – 522 240

April ,2026

Project Description

Startup-Scope-AI is an innovative AI-powered startup evaluation and founder support platform designed to help entrepreneurs validate business ideas, understand market potential, and take strategic next steps with confidence. Many aspiring founders struggle to assess whether their startup concepts are practical, scalable, or competitive. Startup-Scope-AI addresses this challenge by combining artificial intelligence, structured analysis, mentorship support, and event management into one integrated system.

The platform allows users to register securely, submit startup ideas, and receive AI-generated insights based on factors such as feasibility, uniqueness, market demand, competition, and growth opportunities. These intelligent evaluations help founders make informed decisions before investing time and resources. Users can also interact with the system through an AI chat assistant to refine ideas, explore improvements, and gain personalized suggestions.

Beyond idea analysis, the platform offers additional support services including bookmarking promising ideas, booking mentor sessions, and registering for startup-related events. This creates a complete ecosystem where users can move from idea validation to real-world execution and networking.

The system is developed using Flask for backend operations, HTML/CSS/JavaScript for frontend design, and MongoDB / JSON storage for data management. It also integrates modern AI services such as Gemini and OpenAI models to generate intelligent recommendations and analysis.

With its combination of automation, expert support, and user-friendly design, Startup-Scope-AI helps entrepreneurs reduce uncertainty, improve decision-making, and accelerate their startup journey.

Scenarios

Scenario 1: Startup Idea Validation and Market Analysis

A user enters a startup idea into the platform by providing details such as business concept, target audience, industry type, and expected goals. The AI engine analyzes the submitted information and generates a detailed report covering feasibility, market demand, possible competitors, strengths, risks, and growth potential. This helps the founder understand whether the idea is worth pursuing.

Scenario 2: Idea Improvement Through AI Chat Assistance

After receiving the analysis report, the user can interact with the built-in AI chat assistant. The assistant provides suggestions to improve the business model, pricing strategy, customer

acquisition methods, branding ideas, and product features. This interactive guidance helps founders refine their ideas in a practical manner.

Scenario 3: Founder Support Through Mentorship and Events

Once the startup idea is finalized, users can book mentorship sessions with experienced professionals and register for startup events or networking programs. This enables founders to gain expert advice, connect with investors or peers, and move toward successful execution of their startup plans.

Architecture Overview

Startup-Scope-AI follows a modular and scalable full-stack architecture designed to provide efficient startup idea analysis and founder support services. The system uses a Flask-based backend to manage routing, user sessions, business logic, and communication between multiple modules. The frontend is built using HTML, CSS, JavaScript, and Jinja templates, offering a responsive and user-friendly interface.

The platform integrates advanced AI models such as Gemini and OpenAI to process startup ideas, generate analytical reports, and provide conversational guidance through the chat assistant. User data, startup ideas, bookings, chats, and event registrations are stored using MongoDB, with an optional JSON fallback storage system for reliability during connection failures.

The architecture is divided into key modules such as authentication, idea analysis, chat support, bookmarks, mentor bookings, event management, and admin dashboard controls. Each module communicates through secure API routes, ensuring maintainability and smooth data flow across the application.

This structured architecture enables Startup-Scope-AI to deliver fast performance, secure operations, intelligent automation, and future scalability for growing startup communities.

Core Technologies

The successful implementation of Startup-Scope-AI depends on a combination of modern web technologies, backend frameworks, databases, and artificial intelligence services. Each technology plays an important role in delivering efficient performance, secure data handling, and intelligent startup analysis.

HTML, CSS, and JavaScript

These technologies are used to design and develop the frontend interface of the platform. HTML provides the structure of web pages, CSS enhances visual appearance and responsiveness, while JavaScript adds interactivity such as dynamic forms, real-time updates, and API communication.

Flask

Flask is the core backend framework used for developing the application. It handles routing, server-side logic, session management, authentication, template rendering, and communication between the frontend, AI services, and database.

Python

Python is the primary programming language used for backend development. It is used to build APIs, implement business logic, integrate AI models, process user data, and manage system operations efficiently.

MongoDB

MongoDB is used as the primary database for storing user accounts, startup ideas, analysis reports, chat history, bookings, event registrations, and bookmarks. Its flexible document-based structure is ideal for handling dynamic startup-related data.

JSON Storage

A fallback JSON-based storage system is used when the MongoDB connection is unavailable. This ensures that the application can continue operating without data interruption during temporary database issues.

Gemini AI

Google Gemini AI is integrated to analyze startup ideas, generate feasibility reports, provide business suggestions, and assist users through intelligent responses.

OpenAI

OpenAI services are used as an additional AI provider for advanced text generation, startup recommendations, and interactive conversational support.

Jinja Templates

Jinja templating engine is used with Flask to create server-rendered dynamic web pages by combining backend data with frontend HTML templates.

Gunicorn

Gunicorn is used as the production server to deploy the Flask application efficiently in hosted environments.

REST APIs

RESTful APIs are used for communication between frontend components and backend modules such as login, analysis, booking, chat, and admin operations.

Component-Wise Architecture

Component	Description
Frontend UI	Built using HTML, CSS, JavaScript, and Jinja templates. Collects user inputs such as startup idea details, business category, target market, and goals. Displays AI-generated analysis reports, dashboards, chat responses, mentor bookings, event details, and bookmarks in a clean and user-friendly interface.
Flask Backend	Handles core application logic, routing (e.g., /analyze, /chat, /booking, /admin), authentication, session management, and API processing. Acts as the communication bridge between frontend, AI services, and database storage components.
AI Analysis Engine	Uses Gemini AI and OpenAI models to evaluate startup ideas, generate feasibility reports, provide market insights, identify risks, and suggest improvements based on user inputs.
Chat Assistance Layer	Provides conversational support where users can ask follow-up questions, refine startup ideas, receive branding suggestions, and get business growth recommendations interactively.
Mentorship & Event Module	Manages mentor session bookings, startup workshop registrations, networking events, and founder support activities through structured scheduling features.
Bookmark & History Engine	Stores bookmarked startup ideas, previous analyses, comparison reports, and user history for future reference and idea improvement tracking.
Admin Dashboard	Provides admin access for managing users, mentors, startup events, bookings, registrations, and overall platform statistics securely.

Data Storage Layer (MongoDB / JSON)	Stores user profiles, startup ideas, reports, chats, bookings, bookmarks, and event data. JSON fallback storage ensures continuity when database connection is unavailable.
Deployment Layer	Uses Gunicorn and environment configuration files for production deployment, scalability, and secure runtime execution.

5. Pre-requisites

- Python 3.x Installation: https://www.python.org/
- Flask Framework Setup: https://flask.palletsprojects.com/
- MongoDB Installation / Atlas Setup: https://www.mongodb.com/
- Google Gemini API Setup: https://ai.google.dev/
- OpenAI API Access: https://platform.openai.com/
- HTML, CSS, JavaScript Knowledge: https://developer.mozilla.org/
- Jinja Template Basics: https://jinja.palletsprojects.com/
- Gunicorn Installation: https://gunicorn.org/
- Code Editor (VS Code Recommended): https://code.visualstudio.com/

6. Project Workflow

1. AI Integration & Initialization

- Activity 1.1: Create accounts for Google Gemini AI and OpenAI platforms to enable API usage.
- Activity 1.2: Configure API keys, environment variables, access permissions, and test request-response communication.
- Activity 1.3: Design and refine prompt structures for startup idea evaluation, feasibility analysis, market insights, and chatbot assistance.

2. Core Functionality Development

- Activity 2.1: Design Flask backend structure for handling user registration, startup idea submission, AI analysis, and mentor bookings.
- Activity 2.2: Create backend endpoints such as /analyze, /chat, /booking, /bookmark, and /history.
- Activity 2.3: Develop modules for startup scoring, competitor insights, risk analysis, and idea improvement recommendations.

3. Backend Implementation (Flask)

- Activity 3.1: Define Flask routes to manage frontend requests, API responses, and page navigation.
- Activity 3.2: Organize backend into modular components:
  - routes/ : Request handling and API endpoints
  - tools/ : AI logic, startup analysis, and data processing
  - models/ : Schemas and structured validation models
- Activity 3.3: Enable session handling, authentication, and secure access control for users and admin.
- Activity 3.4: Implement MongoDB database connectivity with JSON fallback storage for reliability.

4. UI Development (HTML/CSS/JavaScript)

Activity 4.1: Build structured HTML pages for:

- Landing page with project overview
- User login and registration page
- Startup idea submission page
- AI analysis report dashboard
- Mentor booking and events page

    - Admin management page
- Activity 4.2: Style and format user interface using CSS for responsive layouts, dashboards, cards, and navigation menus.
- Activity 4.3: Use JavaScript for dynamic content loading, API calls, chat interaction, and real-time updates without page reloads.

5. Testing & Optimization

- Activity 5.1: Test workflows using real startup scenarios such as e-commerce ideas, education apps, health tech, and service platforms.
- Activity 5.2: Evaluate AI analysis accuracy, relevance of recommendations, and chatbot response quality.
- Activity 5.3: Tune prompts, validate inputs, and handle incomplete or unclear startup submissions.
- Activity 5.4: Optimize frontend performance, backend response time, and add fallback handling for API or database failures.

MILESTONE 1: Gemini AI Initialization

In this foundational stage, the objective is to establish and configure the core artificial intelligence environment required to power Startup-Scope-AI. The platform uses Google Gemini AI as the primary intelligence engine to analyze startup ideas, generate feasibility reports, compare competitors, and provide strategic recommendations to founders.

This milestone ensures that the development environment is equipped with authenticated API access, proper configuration settings, and seamless communication between the Flask backend and Gemini AI services. Prompt structures are also initialized during this phase to generate accurate and meaningful startup analysis reports.

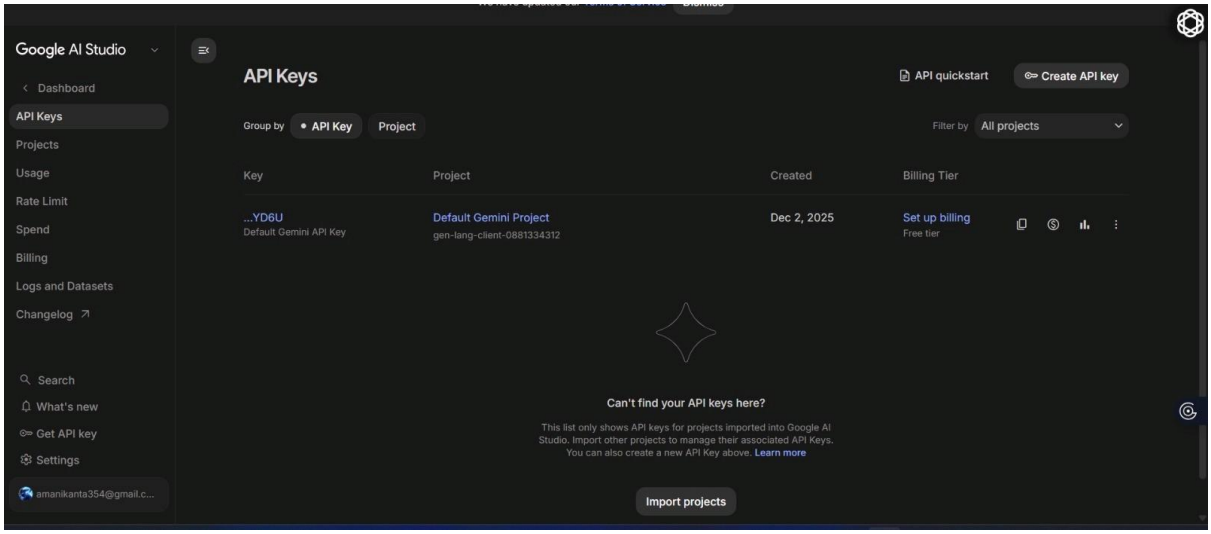
Proper completion of this setup is essential because it enables all AI-driven features such as startup validation, score generation, market opportunity detection, business guidance, and AI consultant support to function reliably throughout the platform.

Activity 1.1: Set up Google AI Studio Account and Enable Gemini API

Sign in or create an account at **https://ai.google.dev/** or Google AI Studio. Enable Gemini API usage for project development.

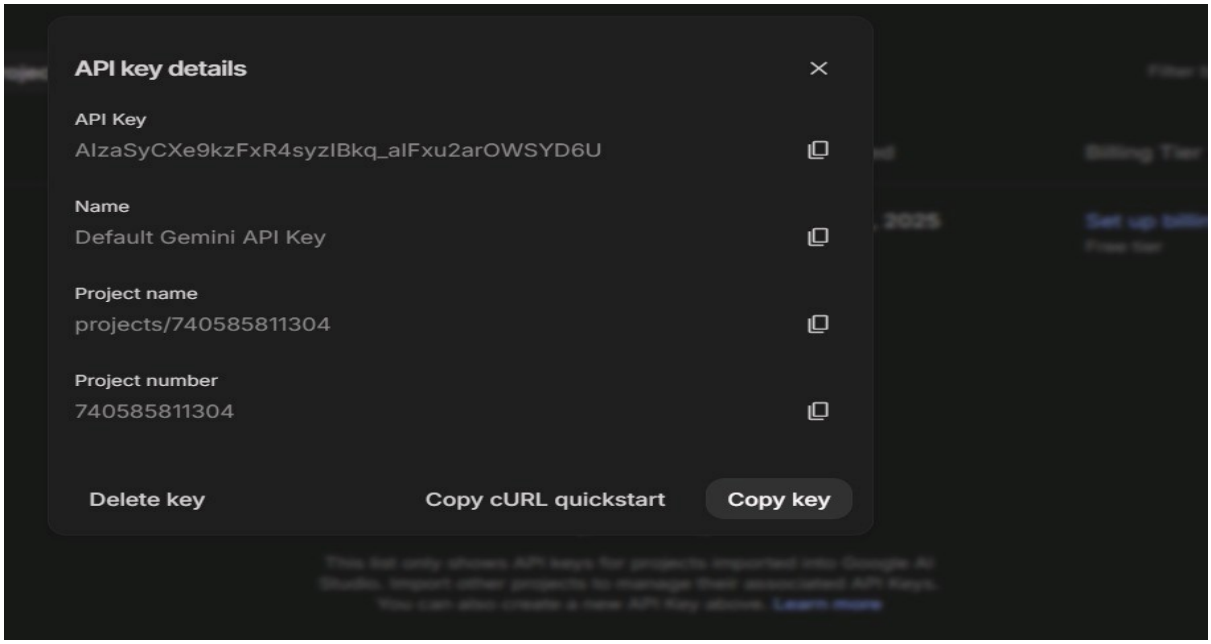
- Open Google AI Studio dashboard.
- Accept all terms and required permissions.
- Create or select a project for Startup-Scope-AI.

- Navigate to API Keys section from the left menu.
- Choose supported Gemini model versions for analysis features.



Activity 1.2: Generate and Configure API Key

- Click on Create API Key button available in the dashboard.
- Generate a new Gemini API key for project integration.
- Copy the generated API key securely.
- Store the key inside the .env file of Flask project.
- Configure environment variables such as:
  GEMINI_API_KEY
  OPENAI_API_KEY (optional)
  SECRET_KEY
- Verify billing tier and quota limits if required.

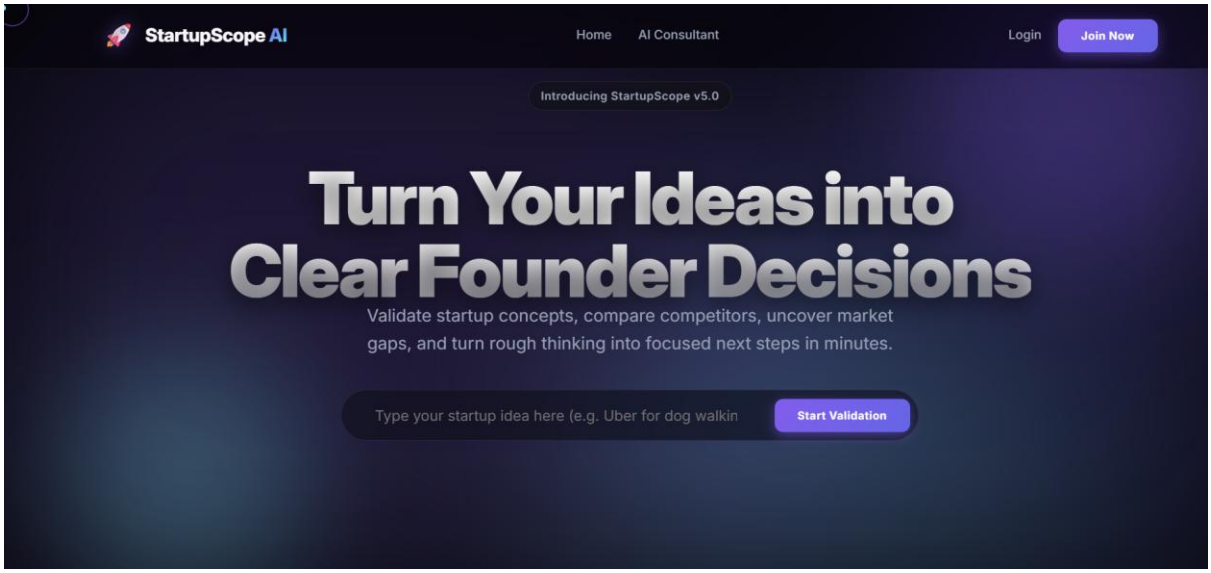


Activity 1.3: Validate API Connectivity Using Python

- Write sample Python code to test Gemini integration.
- Send a startup-related prompt such as:
  "Analyze an online grocery delivery startup idea."
- Check whether valid JSON/text response is returned.
- Test multiple prompts for scoring, competitor insights, and roadmap suggestions.
- Ensure backend can successfully communicate with Gemini AI before proceeding.

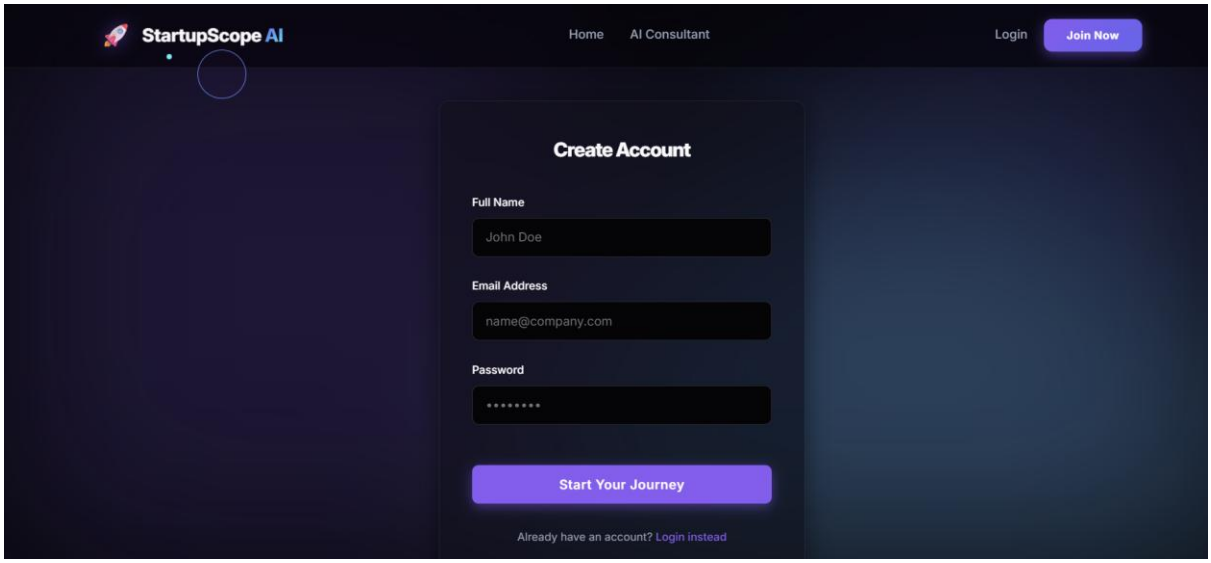
Activity 1.4: Frontend Interface Verification

- Verify landing page of Startup-Scope-AI is loading correctly.
- Check navigation menus such as Home, AI Consultant, Login, and Join Now.
- Validate user registration page design and input forms.
- Confirm dashboard pages open correctly after login.
- Ensure professional UI theme and responsive layout are functioning properly.



Activity 1.5: AI Consultant Module Testing

- Open AI Consultant interface after login.
- Test sample business questions such as strategy planning or roadmap building.
- Validate real-time AI responses for startup founders.
- Ensure message input and send functionality are working correctly.
- Confirm consultant can guide users using previous startup idea context.

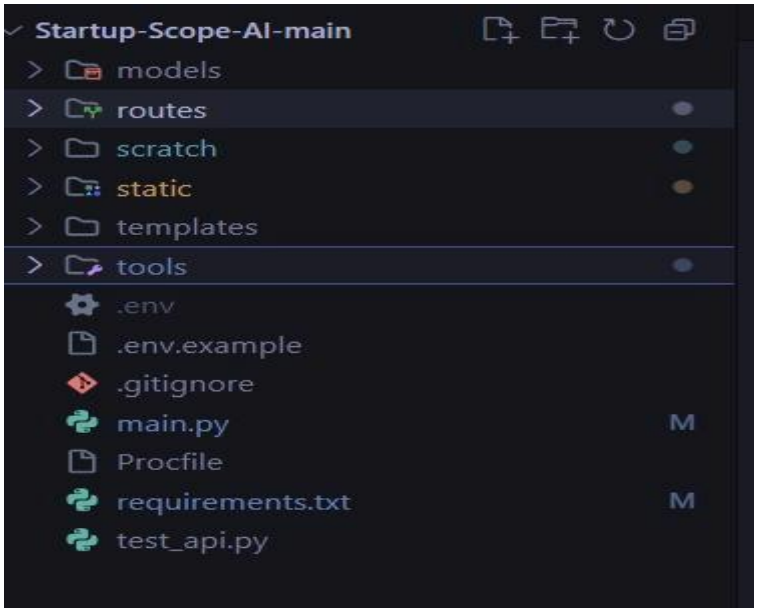


MILESTONE 2: Core Functionalities Development

This milestone focuses on building the core backend intelligence of Startup-Scope-AI using the Flask framework and Google Gemini AI. The primary goal is to develop a robust and modular system capable of accepting startup ideas, target audience details, business descriptions, and founder goals, then intelligently analyzing the information to generate startup feasibility reports and recommendations.

The backend handles prompt generation, API communication with Gemini AI, startup scoring logic, chat assistance, and optional fallback strategies when AI responses are unavailable. All core logic is organized into structured Flask routes and utility modules that manage prompt orchestration, JSON parsing, user sessions, and data retrieval.

File Explorer Structure



Activity 2.1: Flask Initialization and Import Libraries

- Import all required libraries for the project.

```
import os
import json
from flask import Flask, jsonify, render_template, session, redirect, url_for, request, abort
from flask_cors import CORS
from routes.analyze import analyze_bp
from routes.auth import auth_bp
from routes.chat import chat_bp
from routes.bookmark import bookmark_bp
from routes.booking import booking_bp
from routes.admin import admin_bp
from dotenv import load_dotenv
```

- Flask initialization.

```
load_dotenv()

app = Flask(__name__, template_folder="templates", static_folder="static")
```

- Load .env environment variables.

```
# API Keys
OPENAI_API_KEY=sk-proj-qEAZ_qg1fCUuutAbjmCrJRYnYnQb83ahEaPj9Ip5BLX3h0-sYEwd4Ez5Jy0AjjrwcZbJQgAudBT3BlbkF
GOOGLE_API_KEY=AIzaSyCFcl188O7kw-r790_rEuVdhl-1mhDoyD8
FIRECRAWL_API_KEY=fc-a01b75472ccd400d8edcadf0432fa4f3
GROQ_API_KEY=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

# MongoDB Configuration
MONGO_URI=mongodb://localhost:27017/
SECRET_KEY=super_secret_dev_key_123
    Ctrl+L for Agent
```

- Import major modules such as:

```
import os
import json
import re
import requests
from openai import OpenAI
from dotenv import load_dotenv
import google.generativeai as genai
```

Activity 2.2: Main Functionality Code Routings

/analyze : Accepts startup idea details and generates startup validation report.

```python
@analyze_bp.route("/analyze", methods=['POST'])
def analyze_idea():
    try:
        user_id = session.get('user_id')
        data = request.json
        idea_name = data.get("idea_name")
        description = data.get("description")
        target_users = data.get("target_users")

        if not all([idea_name, description, target_users]):
            return jsonify({"error": "Missing required fields"}), 400

        # 1. Search for competitors
        competitors = search_competitors(idea_name, description)

        # 2. Analyze using LLM (V2 logic)
        analysis = analyze_startup(data, competitors)

        # 3. Save to MongoDB/Memory
        record = {
            "idea_name": idea_name,
            "description": description,
            "target_users": target_users,
            "analysis": analysis,
            "created_at": datetime.now()
        }

        save_to_memory(record, user_id=user_id)

        return jsonify(record)
    except Exception as e:
        return jsonify({"error": str(e)}), 500

@analyze_bp.route("/history", methods=['GET'])
def history():
    try:
        user_id = session.get('user_id')
        if not user_id:
            return jsonify({"error": "Authentication required to view history", "needs_auth": True}), 401

        h = get_history(user_id=user_id)
        return jsonify(h)
    except Exception as e:
        return jsonify({"error": str(e)}), 500
```

/**chat** : Provides AI consultant support for founder questions.

```python
chat_bp = Blueprint('chat', __name__)

GOOGLE_API_KEY = os.getenv("GOOGLE_API_KEY")
OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")

@chat_bp.route("/chat", methods=['POST'])
def chat():
    """AI Consultant Chat — follows up on the user's startup analysis."""
    user_id = session.get('user_id')
    if not user_id:
        return jsonify({"error": "Login required"}), 401

    data = request.json
    message = data.get("message", "")
    idea_context = data.get("idea_context", {})
    chat_history = data.get("history", [])

    if not message:
        return jsonify({"error": "Message is required"}), 400

    # Build the system context from the analysis
    system_prompt = f"""You are a startup consultant for StartupScope AI.
The user already validated their startup idea and you are now having a follow-up conversation.

Here is their idea context:
- Name: {idea_context.get('idea_name', 'Unknown')}
- Target: {idea_context.get('target_users', 'Unknown')}
- Description: {idea_context.get('description', 'No description')}
- Score: {idea_context.get('idea_score', 'N/A')}/10

RULES:
- Be direct and practical. No fluff.
- Give specific, actionable advice.
- Keep responses under 150 words unless the user asks for detail.
- Reference their specific idea, not generic startup advice.
"""

    # Build conversation for the API
    messages = [system_prompt]
    for msg in chat_history[-10:]:  # Keep last 10 messages for context
        role = "User" if msg.get("role") == "user" else "Consultant"
        messages.append(f"{role}: {msg.get('content', '')}")
    messages.append(f"User: {message}")

    full_prompt = "\n".join(messages) + "\nConsultant:"
```

/bookmark : Saves favorite startup ideas for later review.

```python
from flask import Blueprint, request, jsonify, session
from tools.memory import toggle_bookmark, get_bookmarks

bookmark_bp = Blueprint('bookmark', __name__)

@bookmark_bp.route("/bookmark", methods=['POST'])
def bookmark():
    user_id = session.get('user_id')
    if not user_id:
        return jsonify({"error": "Login required"}), 401

    data = request.json
    idea_name = data.get("idea_name")
    if not idea_name:
        return jsonify({"error": "idea_name required"}), 400

    result = toggle_bookmark(user_id, idea_name)
    return jsonify(result)

@bookmark_bp.route("/bookmarks", methods=['GET'])
def bookmarks():
    user_id = session.get('user_id')
    if not user_id:
        return jsonify({"error": "Login required"}), 401

    items = get_bookmarks(user_id)
    return jsonify(items)

```

Activity 2.3: User Registration and Health Check Routings

/register : Handles new user registration by accepting and securely storing credentials.

```python
auth_bp = Blueprint('auth', __name__)

@auth_bp.route("/register", methods=['POST'])
def register():
    data = request.json or {}
    name = data.get("name")
    email = (data.get("email") or "").strip().lower()
    password = data.get("password")

    if not name or not email or not password:
        return jsonify({"error": "Name, email, and password required"}), 400

    hashed_password = generate_password_hash(password)
    user_id = create_user(name, email, hashed_password)

    if not user_id:
        return jsonify({"error": "Email already registered"}), 409

    # Automatically log in after registration
    session['user_id'] = user_id
    session['name'] = name
    session['email'] = email
    session['is_admin'] = False

    return jsonify({"message": "Registration successful", "user": {"id": user_id, "name": name, "email": email}})

```

/login : Authenticates user and starts secure session.

```python
@auth_bp.route("/login", methods=['POST'])
def login():
    data = request.json or {}
    email = (data.get("email") or "").strip().lower()
    password = data.get("password")

    if not email or not password:
        return jsonify({"error": "Email and password required"}), 400

    user = verify_user(email, password)
    if not user:
        return jsonify({"error": "Invalid credentials"}), 401

    session['user_id'] = user['id']
    session['name'] = user['name']
    session['email'] = user['email']
    session['is_admin'] = False

    return jsonify({"message": "Login successful", "user": {"id": user['id'], "name": user['name'], "email": user['email']}})

```

/profile : Retrieves and updates account information.

```python
@auth_bp.route("/account/profile", methods=['POST'])
def account_profile():
    user_id = session.get('user_id')
    if not user_id or session.get('is_admin'):
        return jsonify({"error": "Login required"}), 401

    data = request.json or {}
    ok, result = update_user_profile(user_id, data.get("name"), data.get("email"))
    if not ok:
        return jsonify({"error": result}), 400

    session['name'] = result["name"]
    session['email'] = result["email"]
    return jsonify({"message": "Profile updated successfully.", "user": result})
```

Activity 2.4: Ensure Modular Code Structure

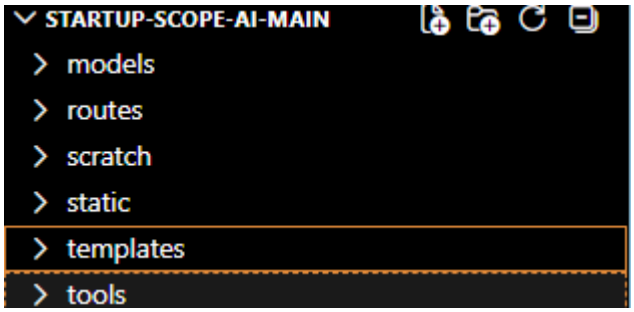
routes/ : Handles endpoint requests and page routing.

tools/ : Contains AI analysis logic, utility functions, memory helpers, and data processing.

models/ : Stores schema definitions and structured validation.

templates/ : HTML frontend pages.

static/ : CSS, JavaScript, and images.



Additional Functional Routes

/compare : Compares two startup ideas and gives recommendation.

Insert screenshot here.

```python
from flask import Blueprint, request, jsonify, session, render_template, redirect, url_for
from tools.memory import (get_all_users, get_all_bookings, get_all_registrations,
                          update_booking_status, update_registration_status, get_db_client,
                          DATABASE_NAME, get_all_mentors, save_mentor, delete_mentor,
                          get_all_events, save_event, delete_event)
from werkzeug.security import generate_password_hash, check_password_hash

admin_bp = Blueprint('admin', __name__, url_prefix='/admin')

# --- Admin Auth ---

def clean_text(value):
    return (value or "").strip()

def is_admin():
    return session.get('is_admin', False)

@admin_bp.route("/login", methods=['GET'])
def admin_login_page():
    if is_admin():
        return redirect('/admin/dashboard')
    return render_template("admin/login.html")

@admin_bp.route("/login", methods=['POST'])
def admin_login():
    data = request.json
    email = data.get("email")
    password = data.get("password")

    # Check against admin collection
    client = get_db_client()
    if client:
        db = client[DATABASE_NAME]
        admin = db["admins"].find_one({"email": email})
        if admin and check_password_hash(admin["password"], password):
            session['user_id'] = admin.get("id", "admin")
            session['name'] = admin.get("name", "Admin")
            session['email'] = email
            session['is_admin'] = True
            return jsonify({"message": "Admin login successful"})

    return jsonify({"error": "Invalid admin credentials"}), 401

@admin_bp.route("/logout", methods=['POST'])
def admin_logout():
```

```python
def admin_logout():
    session.clear()
    return jsonify({"message": "Logged out"})

@admin_bp.route("/dashboard")
def admin_dashboard():
    if not is_admin():
        return redirect('/admin/login')
    return render_template("admin/dashboard.html")

# --- Admin Data APIs ---

@admin_bp.route("/api/stats")
def admin_stats():
    if not is_admin():
        return jsonify({"error": "Unauthorized"}), 403

    users = get_all_users()
    bookings = get_all_bookings()
    registrations = get_all_registrations()

    return jsonify({
        "total_users": len(users),
        "total_bookings": len(bookings),
        "pending_bookings": len([b for b in bookings if b.get("status") == "pending"]),
        "total_registrations": len(registrations),
        "pending_registrations": len([r for r in registrations if r.get("status") == "pending"])
    })

@admin_bp.route("/api/users")
def admin_users():
    if not is_admin():
        return jsonify({"error": "Unauthorized"}), 403
    users = get_all_users()
    # Strip passwords
    for u in users:
        u.pop("password", None)
    return jsonify(users)

@admin_bp.route("/api/bookings")
def admin_bookings():
    if not is_admin():
        return jsonify({"error": "Unauthorized"}), 403
    return jsonify(get_all_bookings())

```

/booking : Allows mentor session booking.

routes > booking.py > ...

```python
from flask import Blueprint, request, jsonify, session
from tools.memory import (save_booking, get_bookings_for_user, get_all_bookings,
                          save_event_registration, get_event_registrations, get_all_registrations,
                          get_registrations_for_user, get_mentor_by_id, get_event_by_id,
                          cancel_booking_for_user, reschedule_booking_for_user,
                          cancel_registration_for_user)
from datetime import datetime

booking_bp = Blueprint('booking', __name__)

def _clean_text(value):
    return (value or "").strip()

@booking_bp.route("/book-session", methods=['POST'])
def book_session():
    user_id = session.get('user_id')
    if not user_id:
        return jsonify({"error": "Login required"}), 401

    data = request.json or {}
    mentor_id = _clean_text(data.get("mentor_id"))
    required_fields = {
        "topic": "Topic",
        "preferred_date": "Preferred date",
        "preferred_time": "Preferred time",
        "session_format": "Session format",
        "message": "Message",
    }
    missing = [
        label for key, label in required_fields.items()
        if not _clean_text(data.get(key))
    ]
    if not mentor_id:
        missing.insert(0, "Mentor")
    if missing:
        return jsonify({"error": f"Please fill: {', '.join(missing)}."}), 400

    mentor = get_mentor_by_id(mentor_id)
    if not mentor:
        return jsonify({"error": "Selected mentor was not found."}), 404

    booking = {
        "user_id": user_id,
        "user_name": session.get('name', ''),
        "user_email": session.get('email', ''),
```

/events : Displays startup events and registrations.

```python
        "user_email": session.get('email', ''),
        "mentor_id": mentor_id,
        "mentor_name": mentor.get("name", data.get("mentor_name")),
        "topic": _clean_text(data.get("topic")),
        "message": _clean_text(data.get("message")),
        "preferred_date": _clean_text(data.get("preferred_date")),
        "preferred_time": _clean_text(data.get("preferred_time")),
        "session_format": _clean_text(data.get("session_format")),
        "status": "pending",  # pending, approved, rejected
        "created_at": datetime.now().isoformat()
    }

    result = save_booking(booking)
    return jsonify({"message": "Booking request submitted!", "booking_id": result})

@booking_bp.route("/register-event", methods=['POST'])
def register_event():
    user_id = session.get('user_id')
    if not user_id:
        return jsonify({"error": "Login required"}), 401

    data = request.json or {}
    event_id = data.get("event_id")
    event = get_event_by_id(event_id)
    if not event:
        return jsonify({"error": "Selected event was not found."}), 404
    if event.get("status") != "upcoming":
        return jsonify({"error": "This event is no longer open for registration."}), 409
    if int(event.get("registered", 0)) >= int(event.get("spots", 0)):
        return jsonify({"error": "This event is already full."}), 409

    existing = [
        item for item in get_registrations_for_user(user_id)
        if item.get("event_id") == event_id and item.get("status") != "cancelled"
    ]
    if existing:
        return jsonify({"error": "You are already registered for this event."}), 409

    registration = {
        "user_id": user_id,
        "user_name": session.get('name', ''),
        "user_email": session.get('email', ''),
        "event_id": event_id,
        "event_title": event.get("title", data.get("event_title")),
```

MILESTONE 3: Backend – Flask Integration (app.py)

This milestone focuses on implementing the Flask-powered backend for **Startup-Scope-AI**. It acts as the bridge between the frontend interface and the AI-based startup analysis engine powered by Gemini and OpenAI. The backend is designed using a modular architecture, providing clean and maintainable routes for startup analysis, chat assistance, user management, bookings, and admin operations.

It also includes features like session handling for personalized user experience, structured API endpoints, and utility modules for smooth AI interaction and reliable frontend rendering. The backend ensures secure data flow, efficient request handling, and scalability of the platform.

Activity 3.1: Define Flask Routes

● **Startup Analysis (/analyze)** : Accepts startup idea details such as business concept, target audience, and goals to generate AI-based feasibility reports. The route processes user input, sends structured prompts to the AI engine, and returns detailed analysis including feasibility score, risks, and recommendations.

● **AI Chat Assistant (/chat)** : Provides conversational support to refine startup ideas, suggest improvements, and answer business-related queries. It maintains context and delivers dynamic responses based on user interaction.

● **Bookmark Management (/bookmark)** : Allows users to save and manage selected startup ideas for future reference. It stores user-specific data and enables easy retrieval of important ideas.

● **History Tracking (/history)** : Retrieves previously analyzed startup ideas and allows comparison between different versions. This helps users track improvements and refine their concepts over time.

● **User Management (/register, /login, /profile)** : Manages user registration, authentication, and profile updates. It ensures secure handling of user credentials and session-based access control.

● **Mentor Booking & Events (/booking, /events)** : Handles mentor session bookings and startup event registrations. It allows users to interact with mentors and participate in relevant startup activities.

● **Admin Dashboard (/admin)** : Provides admin-level access for managing users, mentors, bookings, and platform data. It supports monitoring and control of overall system operations.

Activity 3.2: Modular Architecture Setup

● **routes/** : Contains all route files such as authentication, startup analysis, chat, booking, and admin modules, ensuring separation of concerns.

● **tools/** : Includes AI logic, startup analysis functions, database utilities, and helper methods required for processing requests and generating responses.

● **models/** : Stores schema definitions and structured data validation models to maintain consistency in data handling.

● **templates/** : Contains HTML pages rendered using Jinja templates for dynamic content display.

● **static/** : Includes CSS, JavaScript, and other frontend assets for styling and interactivity.

● **/compare** : Enables comparison between multiple startup ideas and provides recommendation insights based on analysis results.

Activity 3.3: Session Handling & Routing

● Session management is implemented to track logged-in users and maintain authentication state across the application.

● Secure login sessions ensure protected access to user-specific features such as dashboard, bookmarks, bookings, and history.

● Routing connects frontend pages with backend logic using Flask templates and API endpoints, enabling smooth data flow.

● **/dashboard** : Displays user-specific startup insights, recent analyses, and activity overview in a centralized interface.

● **/report** : Shows detailed startup analysis reports generated by AI, including scores, insights, and recommendations.

Activity 3.4: Application Startup and Main Function

● **/** : Renders the landing page with platform overview, features, and navigation options, providing entry point for users.

● **main.py / app.py (main)** : Serves as the entry point of the Flask application. It initializes the app, configures environment variables, sets up session management, and starts the server.

● Registers all blueprints such as authentication, analysis, chat, booking, and admin modules to organize application structure.

● Configures application settings including session handling, CORS policies, and secure key management.

This milestone ensures that all backend components are properly integrated, enabling smooth communication between frontend, AI services, and database systems while maintaining scalability, security, and performance.

no screenshort

MILESTONE 4: UI Development

This milestone focuses on developing a lightweight, responsive, and intuitive frontend for Startup-Scope-AI using HTML, CSS, JavaScript, and Jinja templates. The interface enables users to interact seamlessly with the system's startup analysis, AI chat assistance, mentorship booking, and event management features.

It is tightly integrated with Flask backend endpoints to facilitate real-time input processing and AI-powered output rendering. The goal is to maintain a clean, user-friendly design that supports startup idea submission, displays structured analysis reports, enables chat-based refinement, and provides dashboards for tracking user activities.

Activity 4.1: Develop HTML Frontend Interface

● Design a landing page with navigation to all core features such as Home, AI Consultant, Login, Register, Dashboard, and Events.

Templates Directory Structure:

● Implement individual pages for each module connected to Flask routes.

● Display AI responses in structured card layouts such as:

- Startup analysis reports
- Chat responses
- Booking details
- s Event listings

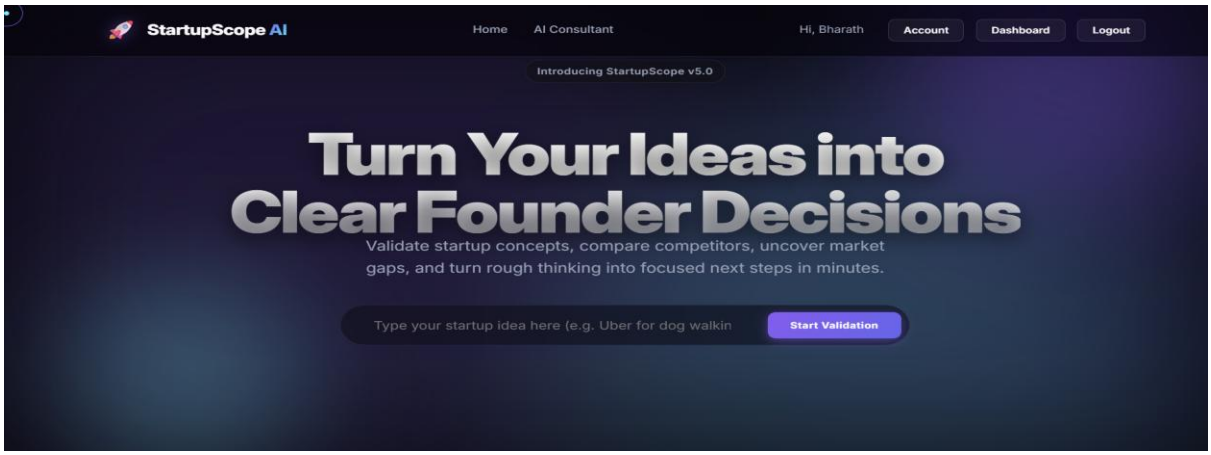
● Ensure consistent layout design across all pages using reusable templates.

● Insert screenshot of templates folder here.

Key UI Pages

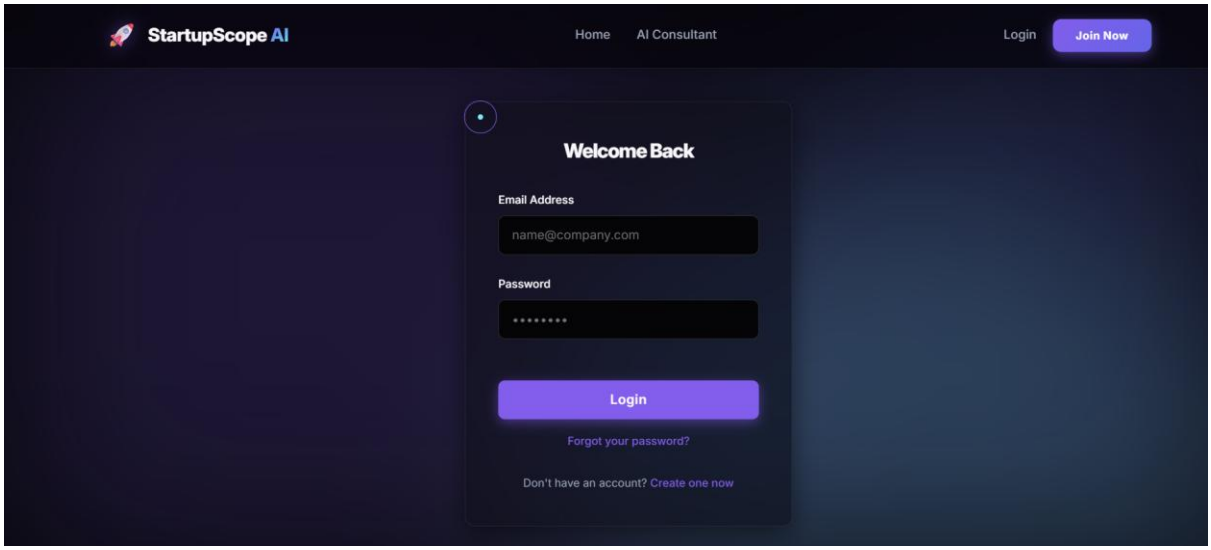
Home Page

Home Page: The main landing page introducing Startup-Scope-AI features such as idea validation, AI consultant, mentorship, and events.



Register / Login Page

User Authentication Page: Allows users to create accounts and securely log in to access personalized features.

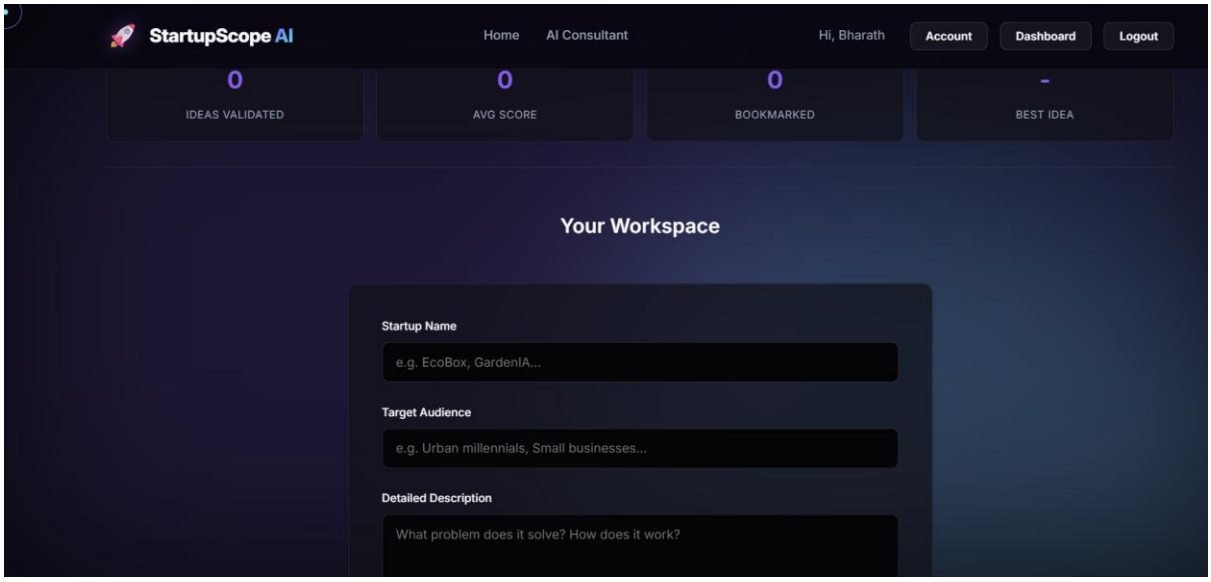


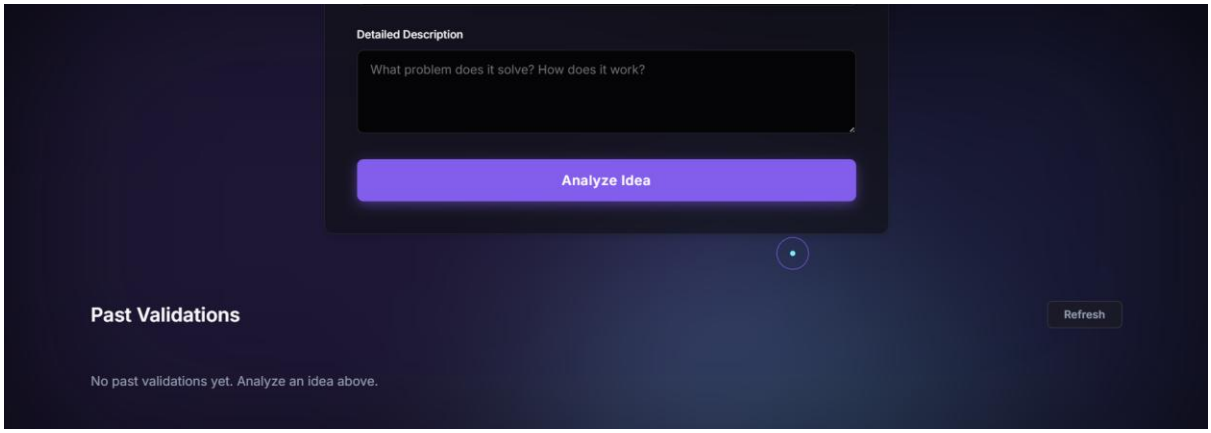
Dashboard Page

User Dashboard: Displays user activity summary, recent startup analyses, bookmarks, and quick access to all features.

Startup Idea Submission Page

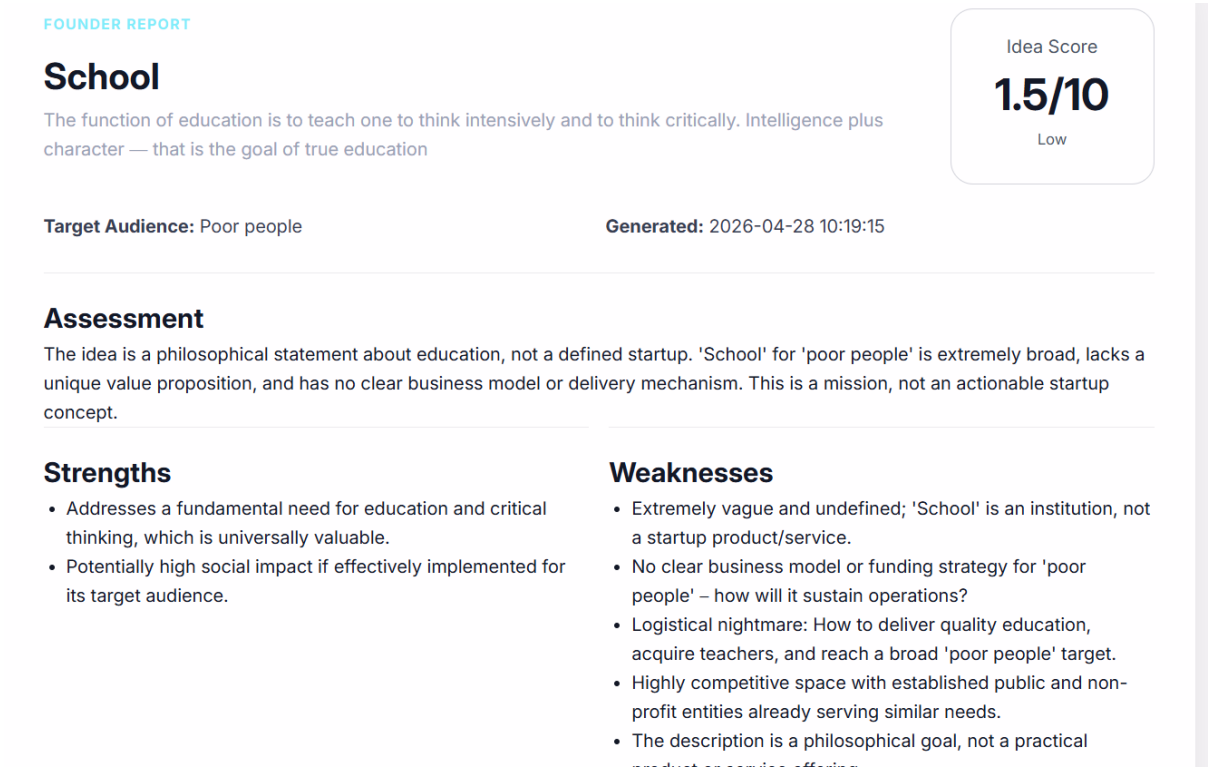
Idea Input Page: Allows users to enter startup idea details such as description, category, target audience, and goals.





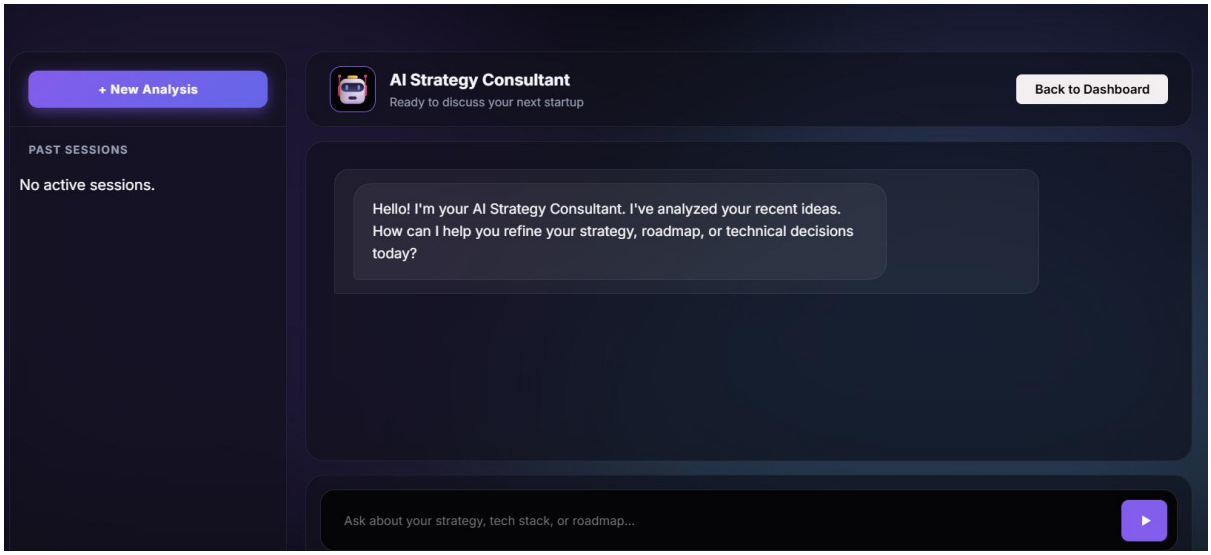
Analysis Report Page

Analysis Report Page: Displays AI-generated startup evaluation including feasibility score, risks, competitor insights, and recommendations.



AI Consultant Page

AI Chat Page: Provides real-time interaction where users can refine ideas, ask business questions, and receive AI suggestions.


+ New Analysis
PAST SESSIONS
No active sessions.
AI Strategy Consultant
Ready to discuss your next startup
Back to Dashboard
Hello! I'm your AI Strategy Consultant. I've analyzed your recent ideas. How can I help you refine your strategy, roadmap, or technical decisions today?
Ask about your strategy, tech stack, or roadmap...

MILESTONE 5: Testing & Optimization

This milestone focuses on validating the reliability, accuracy, and effectiveness of **Startup-S**cope-AI. The system is tested using real-world startup scenarios across different domains such as e-commerce, education, healthcare, and service-based applications to ensure consistent and meaningful AI analysis.

Optimization efforts include refining AI prompt structures, improving response quality, ensuring smooth frontend experience, and handling edge cases effectively.

Activity 5.1: Test with Real-World Startup Scenarios

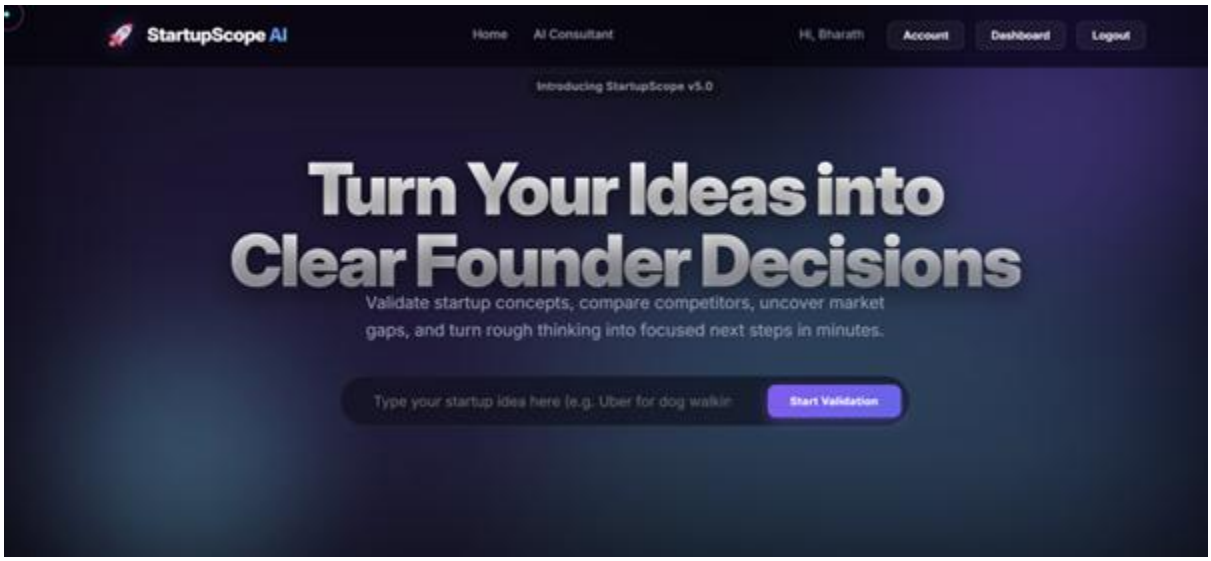
Conduct end-to-end testing using different startup ideas such as:

- E-commerce platforms for online shopping
- Education-based applications for learning
- Healthcare solutions for patient management
- Service-based platforms like booking or delivery systems

UI Testing Pages

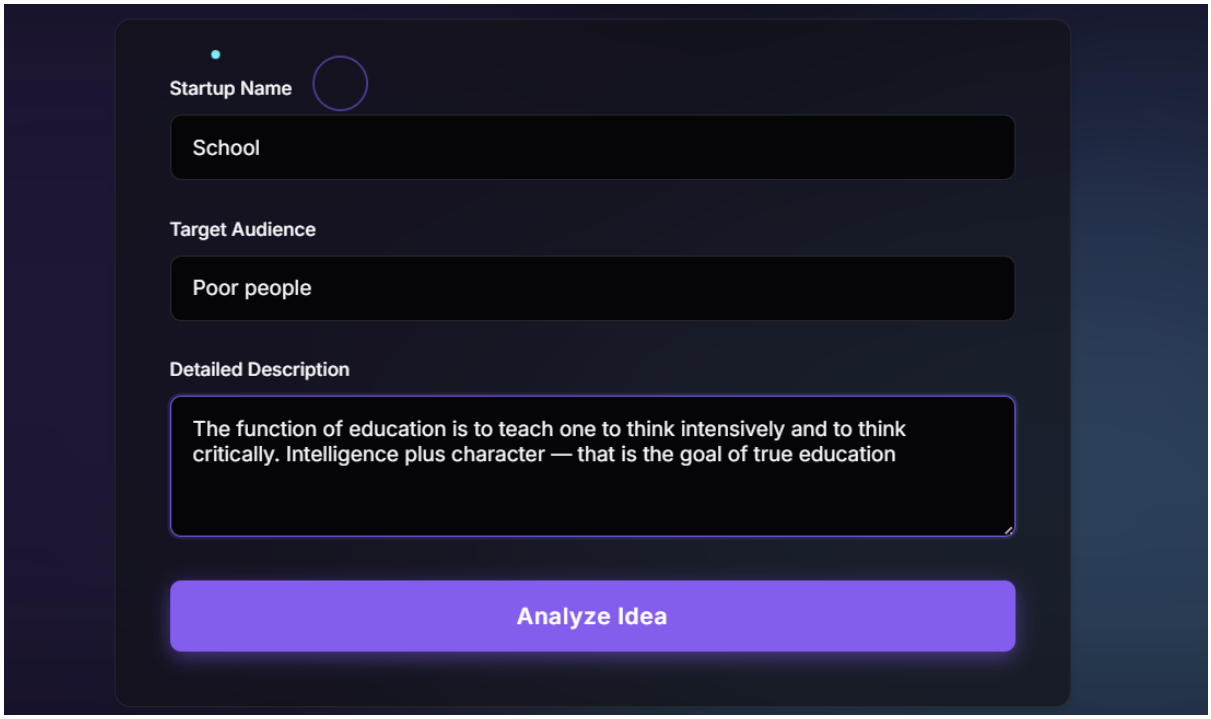
Home Page

Home Page: Verifies navigation, design consistency, and accessibility of core features.



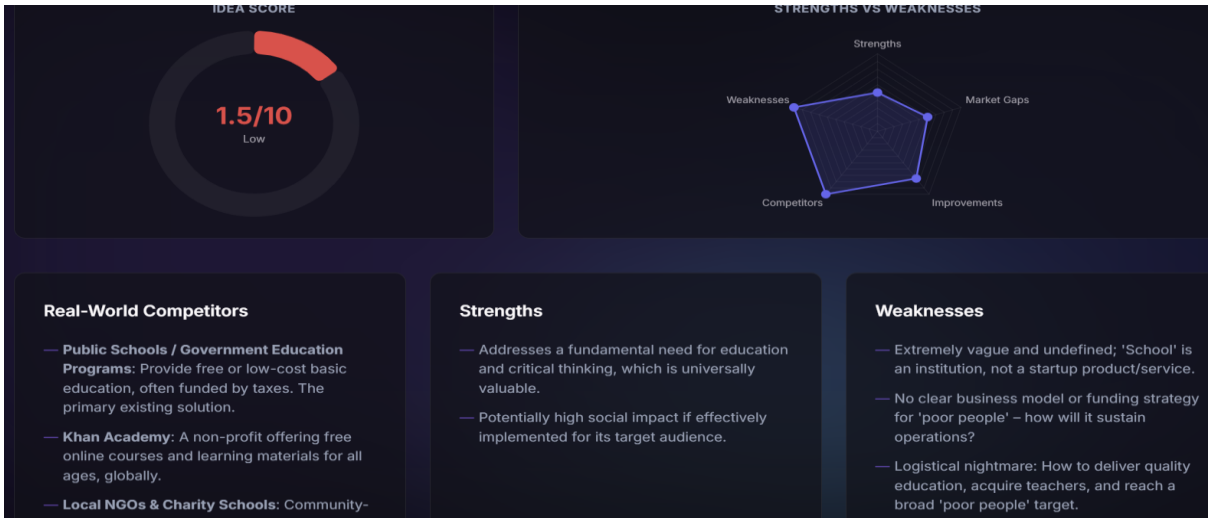
Dashboard Page

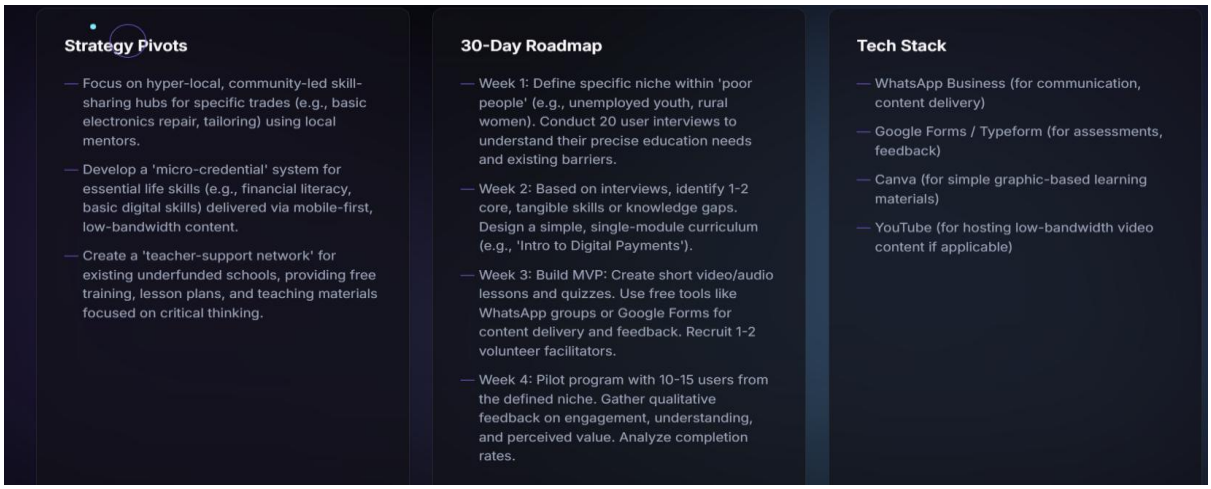
Dashboard Page: Checks user-specific data display such as analysis history and quick navigation.



Idea Submission Page

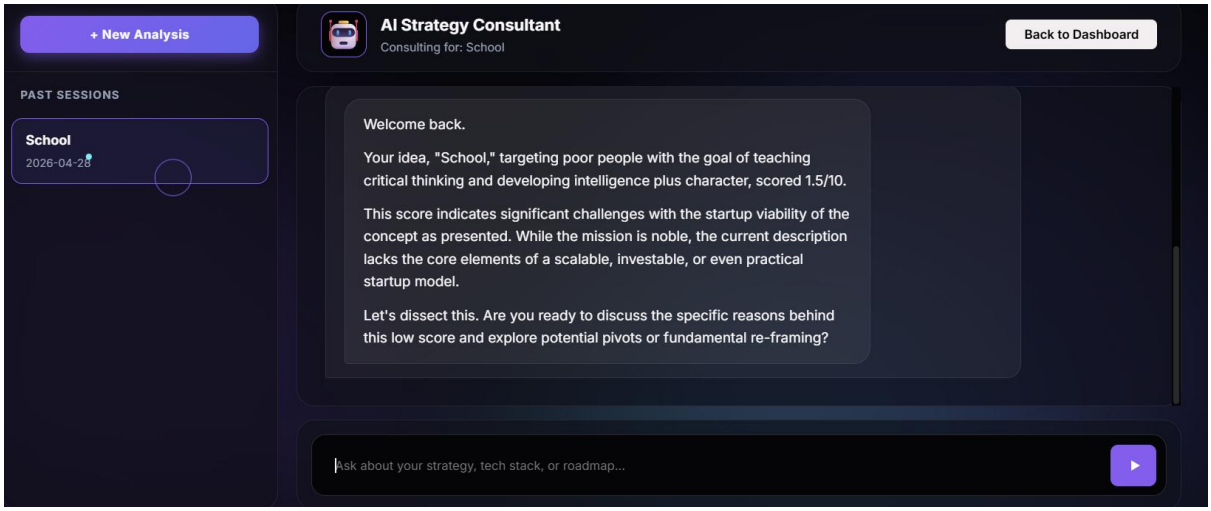
Input Page: Validates form inputs, data handling, and submission functionality.





AI Consultant Page

Chat Page: Tests response accuracy, real-time interaction, and context understanding.

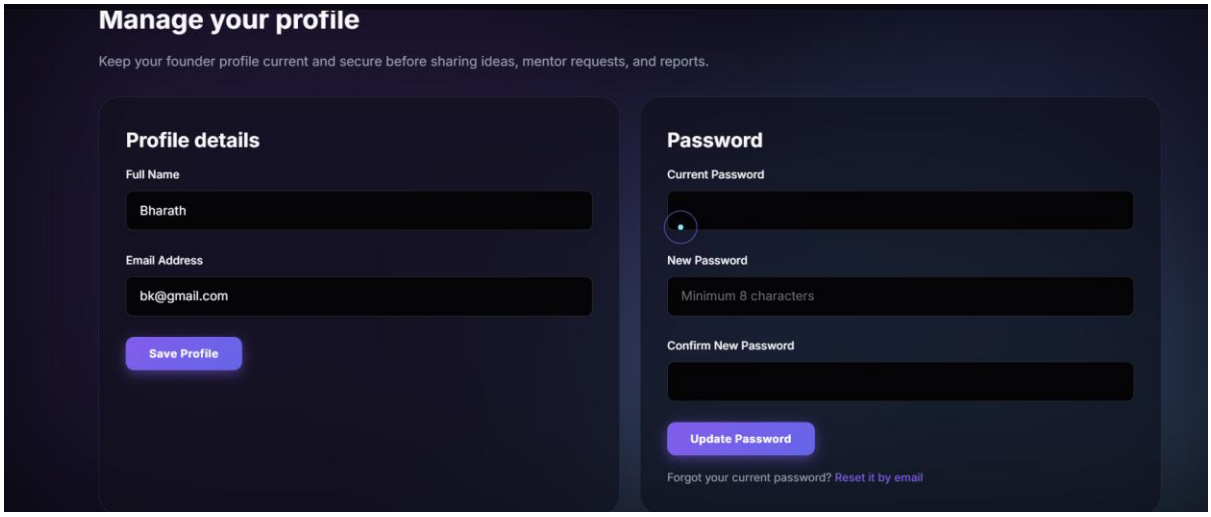


Performance & Optimization

- Evaluate AI response accuracy and relevance.
- Improve prompt quality for better startup insights.
- Optimize frontend responsiveness across devices.
- Handle errors such as invalid input or API failure gracefully.
- Ensure smooth user experience without delays.

Account Page

Account Page: Allows users to manage account details, preferences, and configurations.



This milestone ensures that the system is reliable, efficient, and ready for real-world usage by startup founders.

Conclusion

Startup-Scope-AI successfully demonstrates how artificial intelligence can be integrated with modern web technologies to support startup idea validation and founder decision-making. The platform provides a complete ecosystem where users can analyze their startup ideas, refine them through AI-powered chat assistance, and take further steps such as mentorship booking and event participation.

The system was developed using Flask for backend operations and HTML, CSS, and JavaScript for frontend design, ensuring a responsive and user-friendly interface. Integration with advanced AI models like Gemini and OpenAI enables the platform to generate meaningful insights, evaluate feasibility, identify risks, and suggest improvements in a structured and reliable manner.

Throughout the development process, a modular architecture was maintained, allowing easy scalability, maintainability, and future enhancements. Features such as user authentication, idea history tracking, bookmarking, and admin management add significant value to the platform and improve overall usability.

Testing across various startup scenarios confirmed that the system provides consistent and relevant outputs, helping users make informed decisions before investing time and resources. The combination of AI intelligence, structured workflows, and interactive UI makes Startup-Scope-AI a practical solution for aspiring entrepreneurs.

In conclusion, Startup-Scope-AI serves as an effective and innovative platform that bridges the gap between startup ideas and real-world execution, empowering founders with the tools and insights needed to succeed.